

Clase 3

Los autómatas permiten identificar si una palabra pertenece o no a un determinado lenguaje. Se busca identificar las palabras que tienen un sentido y generan algo.

Los autómatas finitos son mecanismos matemáticos que por medio de iteraciones finitas nos permiten identificar si una palabra pertenece a un determinado lenguaje. Sin embargo, solo se aplican para el reconocimiento de palabras en lenguajes del tipo 3. Por lo tanto, no se aplican a los lenguajes de programación ya que estos son generalmente de tipo 2.

El mecanismo más básico de reconocimiento de palabras es la Máquina de Turing y sirve para los lenguajes de tipo 0.

Las palabras de los lenguajes de tipo 2 se reconocen mediante autómatas finitos denominados autómatas de pila.

Las palabras de los lenguajes de tipo 1 se reconocen mediante autómatas acotados o linealmente acotados.

Los autómatas finitos son un conjunto finito de estados en situaciones constantes que se pueden dar.

Los autómatas finitos están definidos por los siguientes componentes:

$A = (Q, V, \delta, q_0, F)$

$Q \rightarrow$ conjunto de estados $\rightarrow$ por ejemplo, el agua puede estar sólida, líquida o gaseosa.

$V \rightarrow$ alfabeto o vocabulario

$\delta \rightarrow$ funciones de transición $\rightarrow$ cómo se hace para cambiar de estados.

$q_0 \rightarrow$ estado inicial $\rightarrow$ siguiendo con el ejemplo, desde qué estado parte el agua. Debe pertenecer al conjunto de estados.

$F \rightarrow$ estados de los finales $\rightarrow$ a qué estados se puede llegar bajo determinadas condiciones.

Los autómatas finitos se pueden definir de dos formas que son análogas y es posible el pase de uno a otro:

- Tablas de transición
- Diagrama de estados $\rightarrow$ son más difíciles de entender que las tablas pero nos dan mucha más información.

Ejemplo

$A_1 = (\{q_0, q_1, q_2\}; \{x, y\}; \delta; q_0; \{q_1\})$

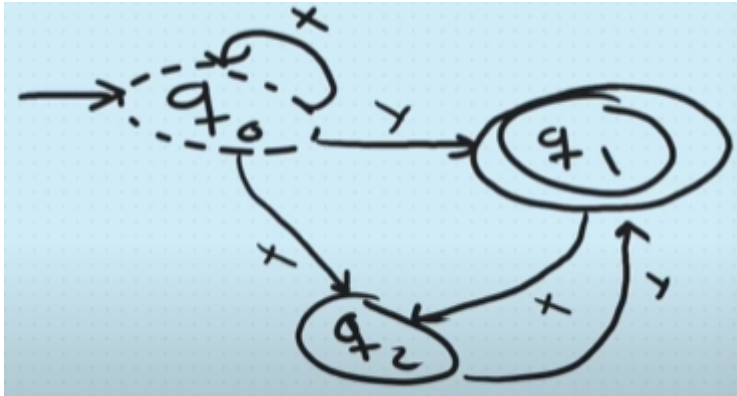
	x	y
q_0	q_0, q_2	q_1
q_1	q_2	-

q_2	-	q_1
-------	---	-------

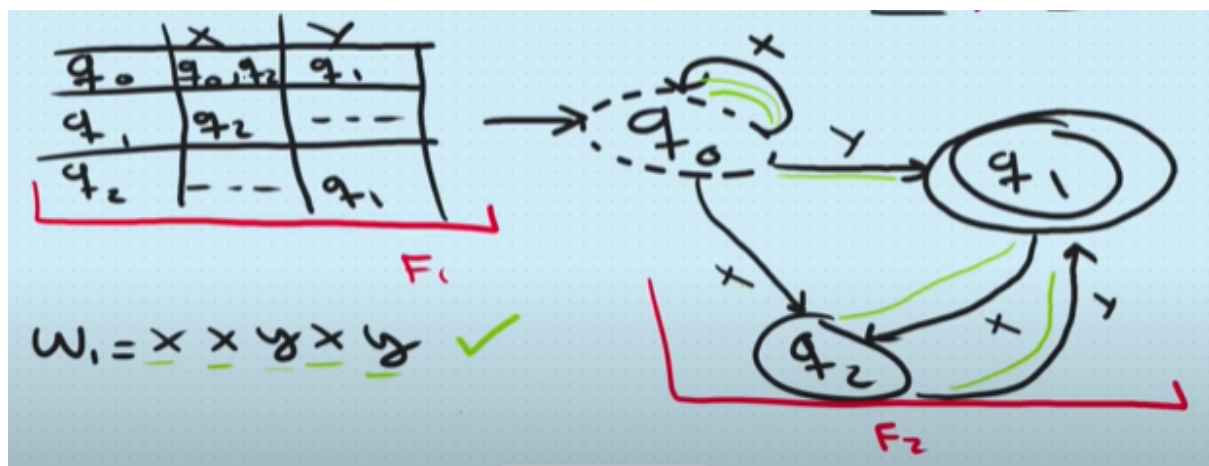
La tabla nos indica dónde estamos y hacia dónde podemos llegar en base a las letras que nos llegan.

Cada uno de los estados son nodos. Sin embargo, al estado inicial se lo suele marcar con línea punteada o una flecha. El estado final se lo suele simbolizar con una doble línea.

A partir de la tabla construimos el diagrama de estados:

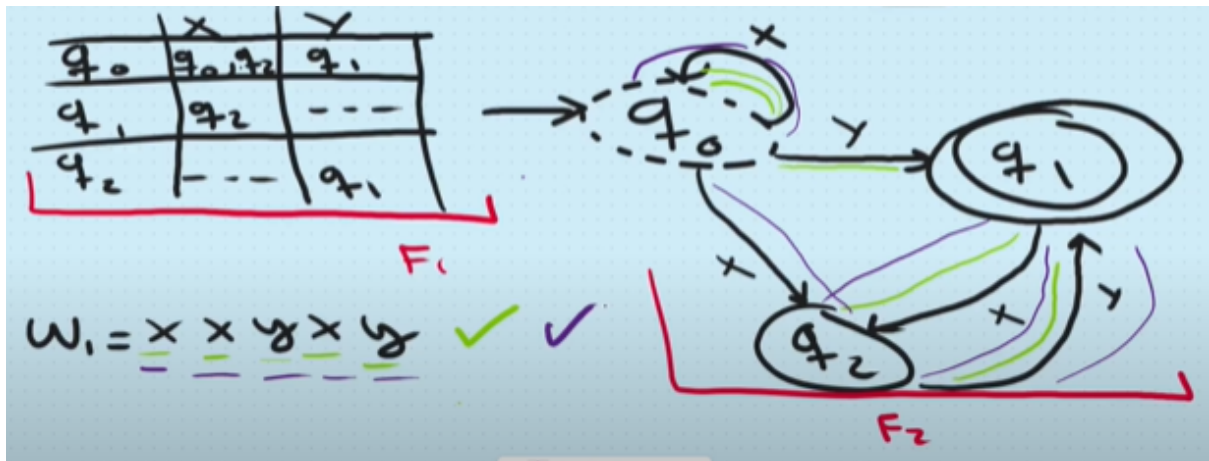


De esta forma, si tenemos la palabra $W_1 = xxyxy$ podemos determinar si pertenece o no al lenguaje. Para eso, vamos viendo si se puede escribir cada una de las letras de izquierda a derecha:



Sin embargo, será una palabra aceptada sólo si termina en un estado final, que en este caso es q_1 , y como la palabra finaliza ahí sí se la considera propia del lenguaje.

No importa el camino que se tome, sino verificar si es posible construir esa palabra:



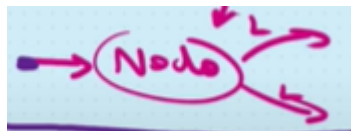
Una palabra que no puede ser aceptada sería $w_2 = yx$, ya que si bien es posible ir uniendo las letras por el diagrama, la palabra no termina en un estado final. En el caso de este diagrama, las palabras aceptadas son aquellas que terminan en y porque son las únicas que te llevan al estado q_1 , el final.

A su vez, los autómatas pueden ser:

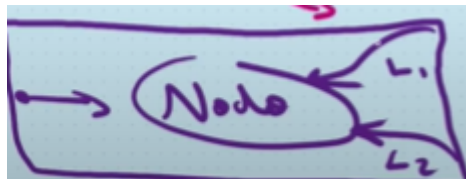
- determinísticos $\rightarrow$ no tiene que haber λ en las transiciones y δ debe cumplir la unicidad, es decir, que desde una letra solo se puede ir a un lugar, no a más de uno. Por lo tanto, dos letras no pueden conducir a más de un nodo.

Es necesario definir el criterio de unicidad que se usa porque existen dos:

- Una letra no puede salir más de una vez de un nodo.

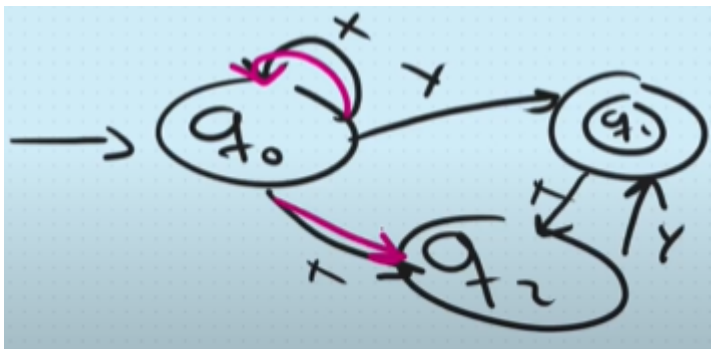


- Una letra no puede llegar más de una vez a un nodo (usamos este)



- no determinísticos

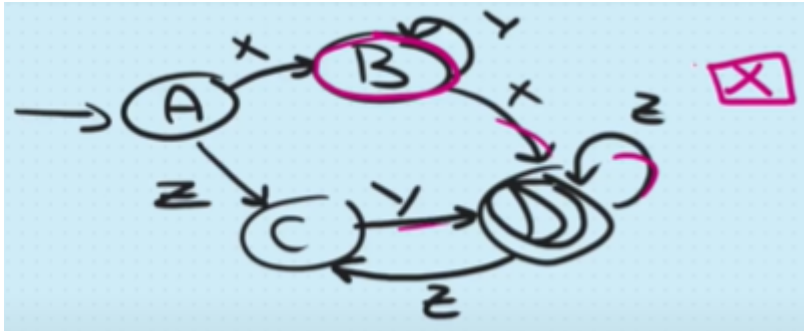
Ejemplo



No es determinístico ya que, si bien no se puede llegar a ningún lado por λ , con la x podemos llegar a dos estados en vez de uno (q_0 y q_2).

Resumiendo, la unicidad se cumple cuando a un nodo ingreso solamente por una letra.

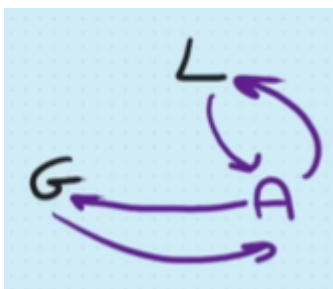
Ejemplos



Es no determinístico porque se puede acceder a los nodos B y D por más de una letra.



Este sí es determinístico.

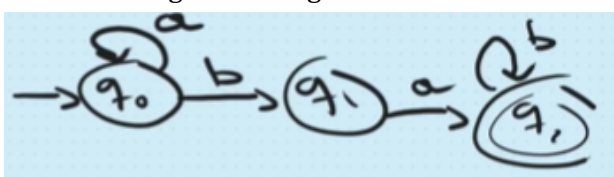


Es posible ir de un autómata a un lenguaje y viceversa, y de un autómata a una gramática y viceversa.

Un autómata tiene un único lenguaje, pero un lenguaje puede tener muchos autómatas.

Autómata a lenguaje

Tenemos el siguiente diagrama de estados:



El lenguaje asociado se obtiene uniendo las letras según el diagrama.
Sería:

a^*bab^*

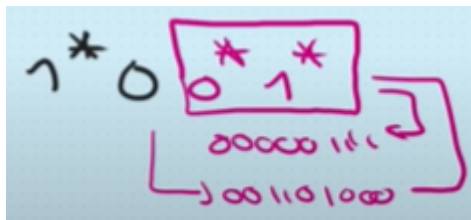
Tenemos la siguiente tabla de transiciones:

	0	1
$\rightarrow a$	b	a
<u>b</u>	b	b

Elaboramos el diagrama de estados:



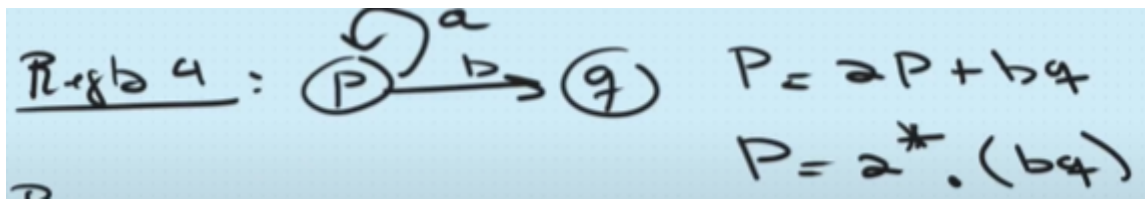
El lenguaje sería:



Sin embargo, la parte rosa hay que ver si representa correctamente lo que indicamos abajo, el intercalado. Nos queda claro que puede haber cero o infinitos 0 y 1 pero no el orden que deben llevar.

Para eso se utilizan las siguientes reglas:





Es un + o un | mientras sigamos expresando de forma cíclica la a que lleva a P, pero cuando pasamos a q mediante b debemos expresar ese fin de ciclo mediante una conjunción.

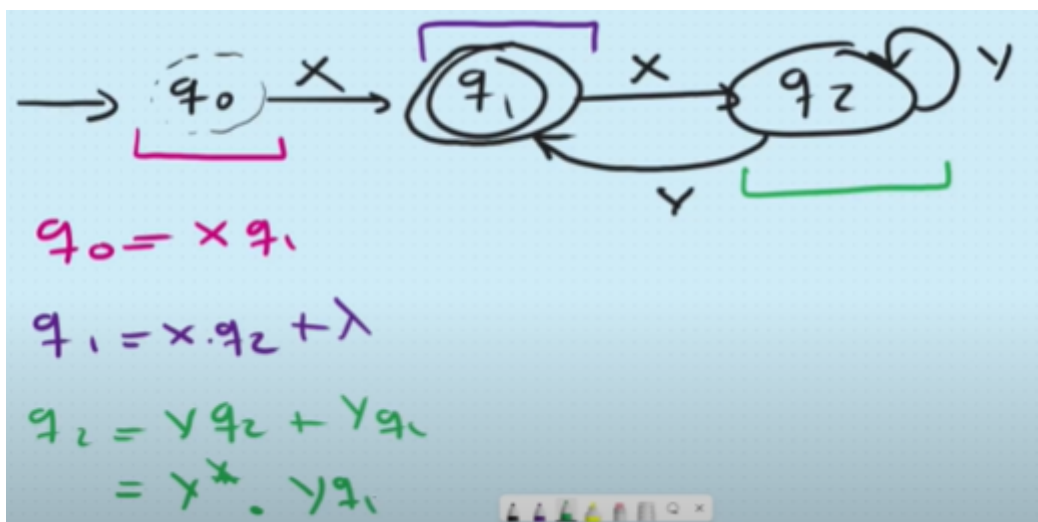


Si estamos en el estado final, es vacío.

Ejemplo de aplicación de reglas para obtener un lenguaje a partir de un diagrama de estados



En primer lugar, debemos evaluar cada nodo por separado evaluando sus salidas para ver cuál regla aplicar.



El segundo paso consiste en armar un sistema de ecuaciones para despejar las q y que queden solo las letras del alfabeto y los lambda. De esta forma, las variables son los estados y las constantes los elementos del vocabulario.

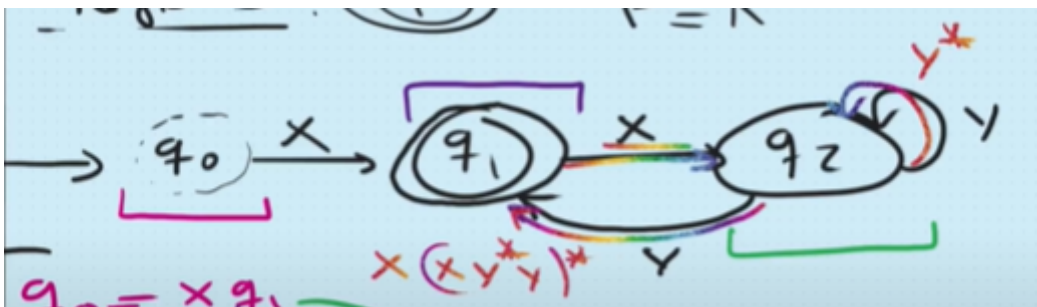
$$\begin{aligned}
 q_0 &= x q_1 \\
 q_1 &= x q_2 + \lambda \quad \rightarrow \quad q_1 = x \cdot (Y^* \cdot Y q_1) + \lambda \\
 q_2 &= Y q_2 + Y q_1 \quad \rightarrow \quad q_1 = (x Y^* Y) q_1 + \lambda \\
 &= Y^* \cdot Y q_1 \quad \rightarrow \quad q_1 = (x Y^* Y)^* \lambda \\
 & \quad \quad \quad \rightarrow \quad q_1 = (x Y^* Y)^*
 \end{aligned}$$

Sustituimos q_2 en la segunda expresión y hacemos cuentas para hallar su valor.
En la tercera cuenta q_1 se simplifica a $*$ porque es el estado final.

Obtenemos q_0 :

$$\begin{aligned}
 q_0 &= x q_1 \\
 q_1 &= x q_2 + \lambda \quad \rightarrow \quad q_1 = x \cdot (Y^* \cdot Y q_1) + \lambda \\
 q_2 &= Y q_2 + Y q_1 \quad \rightarrow \quad q_1 = (x Y^* Y) q_1 + \lambda \\
 &= Y^* \cdot Y q_1 \quad \rightarrow \quad q_1 = (x Y^* Y)^* \lambda \\
 & \quad \quad \quad \rightarrow \quad q_1 = (x Y^* Y)^* \\
 q_0 &= x (x Y^* Y)^*
 \end{aligned}$$

Como el estado inicial es q_0 , el final q_1 y el único camino que los conecta es la x , la palabra mínima es x . Después, se continua por q_1 y vas a q_2 con una x , seguís con todas las y que quieras en q_2 hasta que para volver al estado final (q_1) es necesario otra y .



Cada vez que hay un ciclo (como el que se da entre q_1 y q_2), se representa con una clausura ($*$).

Las reglas facilitan la obtención de la expresión de un lenguaje.

Lenguaje a autómata

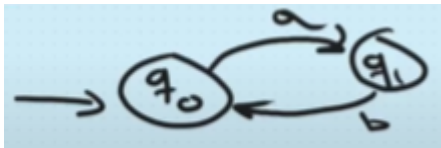
Por ejemplo, el lenguaje:

$$L = \{ (ab)^m c a^P \}$$

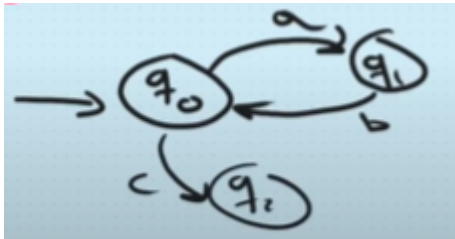
$m \geq 0$
 $P > 0$

Es seguro que va a tener una a porque está P veces.

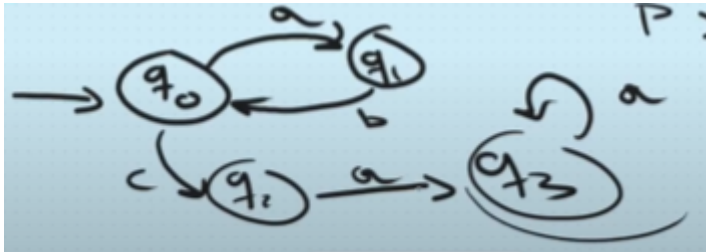
$(ab)^m$ es un ciclo, por lo tanto, tenemos un estado inicial que va a otro estado y vuelve al estado inicial:



Después hay un nuevo estado desde el inicial para moverse a C:



Finalmente, el estado final es la ejecución de a, que puede ser una o infinitas veces:



Si P podía ser 0 había que quedarse en un ciclo en q_2 .

Autómata a gramática

Tenemos el siguiente autómata:

$$A = (Q, N, \delta, q_0, F)$$

$$A = (Q, N, \delta, q_0, F)$$

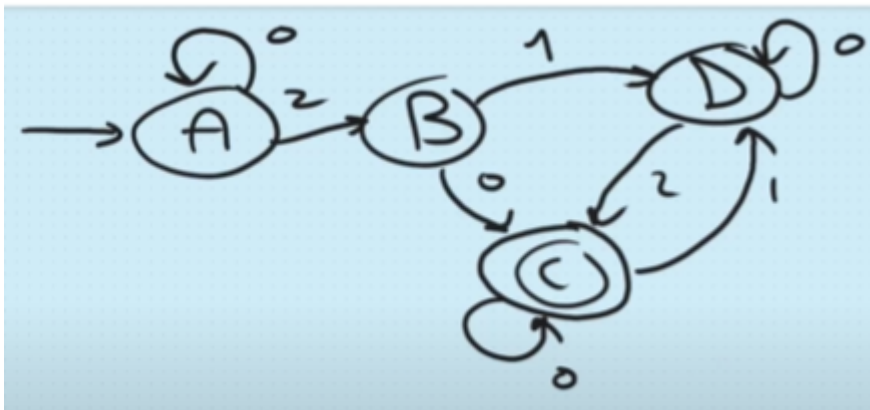
$$\Downarrow$$

$$G = (V_n; V_t; \underline{P}; S)$$

$V_n = Q \quad V_t = V \quad S = q_0$

Las reglas de producción de las gramáticas (P) van a ser el vínculo entre las transiciones (δ) y el estado final (F) de los autómatas. El resto se mapea como indican las flechas rojas.

Ejemplo



Vamos definiendo la gramática a través del diagrama de estados:

$$G = (\{A, B, C, D\}, \{0, 1, 2\}, \underline{P}, A)$$

Solo faltan las producciones, que se obtienen analizando lo que puede derivar cada nodo. Serían:

$$\begin{aligned} A &\rightarrow 0A + 2B \\ B &\rightarrow 1D + 0C \\ C &\rightarrow 0C + 1D + \lambda \\ D &\rightarrow 0D + 2C \end{aligned}$$

C puede derivar en λ porque es el estado final.

Gramática a autómata

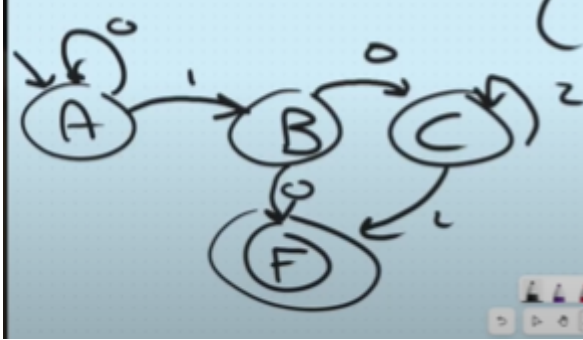
Tenemos la siguiente gramática:

$$G = \{ (A, B, C), (0, 1, 2), P, A \}$$

Con las siguientes reglas de producción:

$$P \begin{cases} A \rightarrow 1B + 0A \\ B \rightarrow 0C + 0 \\ C \rightarrow 2C + 2 \end{cases}$$

El autómata sería:



Se agrega un nodo más que representa el estado final. Por lo tanto, la derivación de $B \rightarrow 0$ te lleva ahí y la de $C \rightarrow 2$ también.